

MIS410 — Week 8 Debrief

Database and SQL | NumPy Internals, Final Critical Thinking Project

Week 8 Overview

Final week of MIS410: a dense reading covering NumPy internals (strides, broadcasting, sorting algorithms, Numba, memory-mapped files), followed by the final critical thinking project — a full SQL + Python pipeline on a fictional e-commerce transactions dataset. This project involved significant real-world debugging around table/column ambiguity and variable-name tracking across a long notebook session.

1. NumPy Internals — Reading Concepts

Strides and the ndarray Structure

An ndarray is fundamentally a flat block of memory plus metadata: a pointer to the data, a dtype, a shape tuple, and a strides tuple (bytes to jump per dimension to move one step). This is why slicing (e.g., `arr[:, :2, ::-1]`) is free — NumPy only changes the stride/starting-point instructions, never copies the underlying data. Negative strides enable walking backward through memory.

Data Type Hierarchy

NumPy dtypes have superclasses (e.g., `np.integer`, `np.floating`) checked via `np.issubdtype(dtype, category)` — a one-category-at-a-time check, not an automatic type identifier. `.mro()` shows a type's full parent chain.

Broadcasting

NumPy's rule for doing math between differently-shaped arrays without manually resizing either one — a smaller array is conceptually "stretched" to match a larger one's shape, without ever copying data. Two shapes are compatible, dimension by dimension from the last dimension backward, if the sizes match or one of them is 1.

Array Combining/Splitting Reference

Function	Description
<code>concatenate</code>	General-purpose join along one axis
<code>vstack</code> / <code>row_stack</code>	Stack by rows (axis 0)
<code>hstack</code>	Stack by columns (axis 1)
<code>column_stack</code>	Like <code>hstack</code> , but converts 1D arrays to 2D column vectors first
<code>dstack</code>	Stack by depth (axis 2)
<code>split</code> / <code>hsplit</code> / <code>vsplit</code>	Split an array along a given axis

Ufunc Methods Reference

Method	Description
<code>accumulate(x)</code>	Running total — keeps every intermediate step
<code>reduce(x)</code>	Collapses the array to a single final value
<code>at(x, indices, b)</code>	Applies an operation in-place at specific index positions
<code>reduceat(x, bins)</code>	"Group by"-style reduce — collapses contiguous segments separately
<code>outer(x, y)</code>	Applies the operation to every pair of elements between x and y

Indirect Sorts — `argsort` and `lexsort`

`argsort()` doesn't sort data directly — it returns an array of index positions describing how to visit the data in sorted order (a "map," not the sorted values themselves). Applying that index array to the original data produces the actual sorted result. This indirection lets a sort order computed from one array be applied consistently across other related arrays/rows. `lexsort()` extends this to multiple sort keys — keys are passed in reverse priority order, so the last argument is the primary sort key.

Sorting Algorithm Comparison

Kind	Speed	Stable	Work space	Worst case
quicksort	1 (fastest typical)	No	0	$O(n^2)$
mergesort	2	Yes	$n/2$	$O(n \log n)$
heapsort	3 (slowest typical)	No	0	$O(n \log n)$

Stability = whether equal elements keep their original relative order after sorting. Work space = extra memory needed beyond the original array (0 = sorts in place; $n/2$ = needs roughly half again as much extra space). Mergesort is the standard choice when tie-breaking order matters; NumPy defaults to quicksort.

Numba — Custom ufuncs

The `@vectorize` decorator from Numba compiles a plain, simple Python function into a fast, true NumPy ufunc — automatically gaining ufunc bonus methods like `.accumulate()` for free. Core takeaway: write simple Python, get NumPy-level speed without touching a compiled language directly.

Memory-Mapped Files

`np.memmap` lets NumPy interact with a huge file on disk as if it were an in-memory array, without loading the whole thing into RAM — only the small piece being actively used is touched. Changes are buffered until `.flush()` writes them back to disk. Reopening a memmap later still requires specifying dtype and shape, since the raw file itself carries no type/shape metadata. Specialized tool for datasets too large to fit in memory; not typically needed for this course's assignment scale.

Structured/Nested Data Types

A structured dtype can give a named field its own shape (e.g., a field that is itself a 3-element sub-array per record), or nest one structured type inside another. Flagged as advanced/rare — understood conceptually, not expected to be used regularly; pandas does not support this exact nested-field pattern (closest analog is

hierarchical indexing).

2. Critical Thinking Project — E-Commerce SQL + Python Pipeline

Dataset: fictional e-commerce transactions, spread across two tables — `payment_method` (`transaction_id`, `customer_id`, `product_id`, `transaction_date`, `purchase_amount`, `payment_method`) and `inventory` (`product_id`, `category`, `price`, `launch_date`).

Task 1 — SQL Queries

1. All 2024 transactions:

```
SELECT transaction_id, customer_id, product_id, transaction_date, purchase_amount
FROM payment_method
WHERE transaction_date BETWEEN '2024-01-01' AND '2024-12-31'
```

2. Electronics category transactions (required a JOIN, since category lives in inventory):

```
SELECT transaction_id, customer_id, inventory.product_id, purchase_amount
FROM payment_method
JOIN inventory ON payment_method.product_id = inventory.product_id
WHERE inventory.category = 'electronics'
```

3. Total purchase amount per customer:

```
SELECT customer_id, SUM(purchase_amount) AS total_spent
FROM payment_method
GROUP BY customer_id
```

4. Product purchase count by category:

```
SELECT inventory.category, COUNT(inventory.product_id) AS purchase_count
FROM payment_method
JOIN inventory ON payment_method.product_id = inventory.product_id
GROUP BY category
```

Task 2 — Python Analysis

Data loading/cleaning: loaded a full joined dataset via SQLAlchemy; confirmed zero missing values across all columns.

Descriptive statistics: mean, median, and standard deviation calculated for `purchase_amount`; distribution visualized with a histogram — purchase amounts ranged roughly \$0–\$3,000 in a fairly flat/uniform shape rather than a typical bell curve.

Sales by category: aggregated and visualized with a bar chart — Books had the highest total sales by a clear margin; Clothing was the lowest.

Customer segmentation: customers grouped into Low (<\$500), Medium (\$500–\$1,000), and High (>\$1,000) spending tiers using `pd.cut()` on per-customer total spend, visualized with a pie chart.

Task 3 — Combining SQL and Python

Monthly sales trend: extracted month from `transaction_date`, aggregated total `purchase_amount` per month, visualized as a line plot. Result: a clear upward trend across the year, from roughly \$150 in month 1

to over \$1,100 by month 12.

Price vs. purchase amount correlation: joined inventory.price to payment_method.purchase_amount on product_id. Result: correlation ≈ 0.01 — essentially zero. Likely explanation: purchase_amount reflects total transaction value (quantity, taxes, discounts), not a single unit's price, so no clean price-to-amount relationship would be expected.

Payment method behavior: average purchase_amount grouped by payment_method, visualized as a bar chart. Result: Apple Pay highest (~\$1,606), Google Pay and Debit Card lowest (~\$1,465–\$1,468) — differences across methods were modest overall.

Task 4 — Summary Report (submitted)

Covered methodology (SQL retrieval + Python pipeline), interpretation (payment method shows minimal difference in purchase behavior; sales trending up monthly; price and purchase amount show no correlation; segmentation shows customers who buy tend to spend heavily), business insight (payment method is not a purchasing obstacle; growth driver unidentified and worth further investigation; price is not a lever for transaction size), and challenges (table/column ambiguity, and variable-name tracking across a long notebook session).

3. Key Debugging Lessons

- **Column ambiguity after a JOIN.** Once two tables are joined, any column existing in both (like `product_id`) must be prefixed with its table name (`table.column`) in both the `SELECT` and any aggregate function — this came up multiple times and was consistently the fix.
- **A CSV's filename is unrelated to its database table name.** A file saved locally as `inventory.csv` only becomes the table `inventory` because that's the name explicitly passed to `.to_sql()` — not because of the filename. This caused real confusion when a table name from a previous week's project (`products`) was assumed to still apply.
- **Reused variable names caused most of the session's friction.** Reusing names like `df_2024` and `query_full` across several different, unrelated queries meant repeatedly losing track of which data currently lived in a variable — several "wrong answer" moments were really "correct code, wrong variable currently holds different data." Worth building a habit of content-descriptive variable names going forward.
- **SUM vs. COUNT vs. AVG mixing.** Recurring confusion between these three aggregate functions — reinforced distinction: `COUNT` = how many rows, `SUM` = total of a numeric value, `AVG` = typical value.
- **New tool anxiety (`pd.cut`).** A brand-new tool introduced for the first time (bucketing continuous values into labeled categories) initially felt overwhelming, but was picked up quickly once broken into two clear steps: aggregate first, then bucket.

Reflection

The statistical and logical reasoning was consistently solid all week — every genuine struggle traced back to environment/bookkeeping issues (table names, variable names, column ambiguity) rather than misunderstanding the underlying concepts. This is a meaningful distinction: the debugging skill being built here — systematically checking "which table has this column" and "what does this variable currently hold" — is exactly the kind of real-world troubleshooting that doesn't show up in textbook reading but is a core, transferable skill for actual data analyst work.